type. It can represent a complex type, so if your template is designed to work with more than just a list of cities, you can create a wrapper type that includes everything your template needs.

There is an alternative that enables you to define a template that can take any number of parameters. To take advantage of this feature, create a method in a code or functions block that returns `void` (or `Task`, if async processing is required), and just include HTML markup in the body of the method.

**Listing 3.13 Alternative way to declare a template within a Razor page**

```
@{
   void MyUnorderedListTemplate(List<City> cities, string style)     ❶
    {
        <ul>                                                         ❷
        @foreach(var city in cities)
        {
            <li class="@(city.Name == "London" ? style : null)">@city.Name</li>
        }
        </ul>
    }
}
@{ MyUnorderedListTemplate(cities, "active"); }                      ❸
```

❶ Markup is allowed in methods that return void or Task.

❷ The opening tag is not prefixed with the @ symbol.

❸ Since the method returns void or Task, it must be called within a code block.

Notice that, unlike the template delegate, the method can specify any number of parameters, and the leading HTML tag is not prefixed with the `@` symbol. This method takes two parameters—the second one representing the name of a CSS class that should be applied conditionally to the list items. If you wanted to be able to achieve something similar with a template delegate, you would need to create a new type that wrapped the `List<City>` and the `string`.

Whether you use this approach or the template delegate, these helpers are only intended for reuse within the same Razor page. If you want to reuse snippets of HTML in multiple pages, there are much more flexible alternatives, including partial pages, tag helpers, and view components. We will look at partial pages and